آزمون جهش کم‌هزینه با پیش‌بینی یادگرفته‌شده

Predictive Mutation Testing at Scale

تیم ۸ · ارائه‌ی روش، baselineها و نتایج pilot

مسئله: آزمون جهش معیار خوبی است ولی گران

- آزمون جهش (mutation testing) خطاهای کوچک عمدی (mutant) در کد می‌کارد و می‌سنجد تست‌ها چندتا را «می‌کشند».
- نمره‌ی جهش = تعداد mutantهای کشته‌شده ÷ تعداد کل mutantها؛ معیاری قوی‌تر از پوشش کد.
- هزینه: هر mutant باید با همه‌ی تست‌های پوشش‌دهنده اجرا شود؛ در پروژه‌های واقعی هزاران mutant داریم.
- **سؤال: آیا می‌توان بیشتر نتیجه‌ها را پیش‌بینی کرد و فقط درصد کمی را واقعاً اجرا کرد، بی‌آن‌که دقت زیاد کم شود؟**

2,451

mutant در 10 پروژه‌ی Java

4 min

زمان PIT با ماتریس کامل (۴ هسته)

5–20%

بودجه‌ی اجرای مجاز در روش ما

ایده و سؤال‌های پژوهشی

ایده: مدل یادگیری ماشین کالیبره‌شده احتمال کشته‌شدن هر mutant را پیش‌بینی می‌کند؛ یک سیاست فعال، آموزنده‌ترین mutantها را برای اجرای واقعی انتخاب می‌کند؛ و یک تخمین‌گر ناریب، نمره‌ی جهش را با بازه‌ی اطمینان ۹۵٪ می‌دهد.

RQ1

دقت پیش‌بینی

Precision، Recall، MCC و AUC؛ درون‌پروژه‌ای و بین‌پروژه‌ای

RQ2

دقت تخمین نمره

MAE در بودجه‌های ۵، ۱۰ و ۲۰٪ و پوشش بازه‌ی اطمینان

RQ3

صرفه‌جویی

درصد mutant و اجرای تست، و زمان صرفه‌جویی‌شده

RQ4

کاربرد

رتبه‌بندی مجموعه‌تست‌ها (Kendall τ)

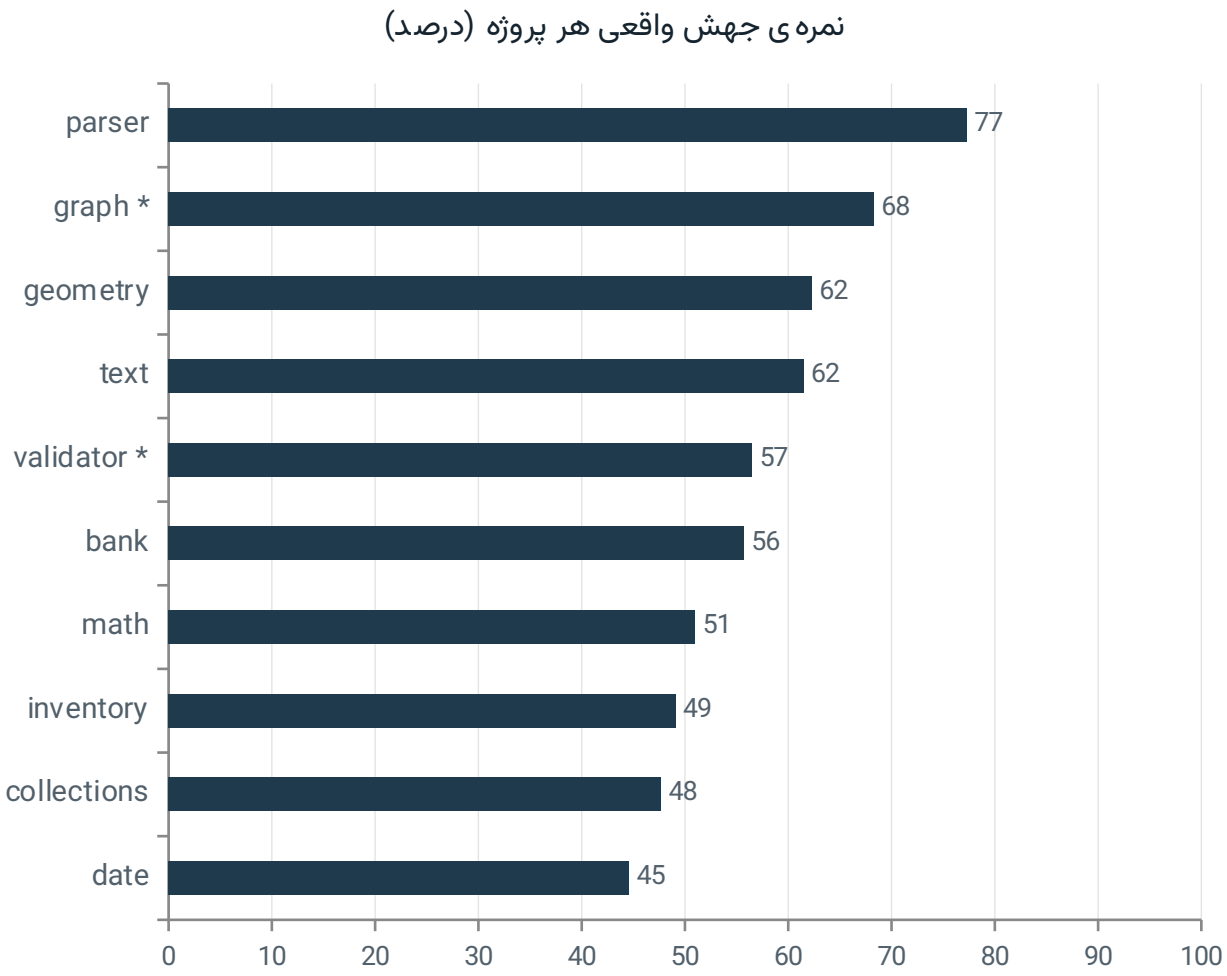
RQ5

Ablation

حذف گروه‌های ویژگی و اجزای روش

داده: ۱۰ پروژه‌ی Java و ماتریس کامل PIT

- ۱۰ پروژه‌ی کوچک Java با تست JUnit 5 (بانک، هندسه، متن، ساختمان‌داده، ریاضی، تاریخ، parser، انبار، گراف، اعتبارسنجی).
- PIT 1.30.1 با fullMutationMatrix و همه‌ی عملگرها: 2,451 mutant، برای هر mutant فهرست تست‌های کشنده.
- ۸ پروژه برای آموزش و ارزیابی leave-one-project-out؛ ۲ پروژه‌ی ستاره‌دار (graph و validator) تا ارزیابی نهایی دست‌نخورده می‌مانند.
- پروژه‌های واقعی Apache Commons در java/subjects.csv آماده‌اند و با یک دستور اضافه می‌شوند.



ویژگی‌های هر mutant (بدون اجرای آن)

1 عملگر

نوع عملگر PIT و خانواده‌ی آن (شرط، ریاضی، return، حذف فراخوانی، ثابت...)

2 ساختار کد

نوع دستور، عمق تودرتو، اندازه و پیچیدگی متد، موقعیت در متد، اندازه‌ی کلاس، تعداد mutant هم‌خط

3 پوشش

تعداد تست‌های پوشش‌دهنده و کلاس‌های تست (از فاز coverage در PIT، پیش از اجرای mutant)

4 قدرت تست (oracle)

تعداد assertها، تست‌هایی که متد جهش‌یافته را مستقیم صدا می‌زنند یا روی آن assert دارند

Baselineها: همه با همان داده و همان بودجه

Baseline	چیست	اجرای mutant
اجرای کامل PIT	مرجع طلایی، برای محاسبه‌ی خطا	۱۰۰٪
نمونه‌گیری تصادفی	اجرای تصادفی به اندازه‌ی بودجه، بازه‌ی اطمینان با تصحیح جمعیت محدود	بودجه
نمونه‌گیری طبقه‌ای	طبقه‌بندی بر اساس خانواده‌ی عملگر، تخصیص متناسب	بودجه
جهش انتخابی (Selective)	فقط چند عملگر پرکاربرد تا سقف بودجه	حداکثر بودجه
PMT (Zhang و همکاران)	Random Forest با ویژگی‌های PMT، پیش‌بینی همه بدون اجرا	۰٪
Seshat-lite	تقریب Seshat: توکن‌های خط جهش‌یافته + رگرسیون لجستیک	۰٪
فقط مدل / Active top-k	جمع احتمال‌های کالیبره / اجرای نامطمئن‌ترین‌ها به اندازه‌ی بودجه	۰ / بودجه

نام و نسخه‌ی PMT و Seshat را ابتدای کار با مقاله و مخزن اصلی تطبیق دهید؛ اینجا بازپیاده‌سازی شده‌اند.

روش پیشنهادی ACE-MT: نمای کلی

Active, Calibrated, model-assisted Estimation of Mutation score



نوآوری: گام‌های ۳ تا ۵ با هم. انتخاب فعال تخمین را اریب نمی‌کند، چون احتمال انتخاب هر mutant معلوم است و فقط به برچسب‌های قبلی بستگی دارد.

گام ۱ و ۲: مدل بین‌پروژه‌ای کالیبره و اطلاعات رایگان

- **آموزش روی پروژه‌های دیگر (leave-one-project-out)؛ پروژه‌ی هدف هیچ برچسبی به مدل نمی‌دهد.**
- مدل: XGBoost (قابل تعویض با RF، LightGBM و LogReg از فایل YAML).
- کالیبراسیون isotonic با foldهای گروه‌بندی‌شده بر اساس پروژه؛ یعنی «احتمال ۰٫۷» واقعاً حدود ۷۰٪ کشته‌شدن باشد.
- بدون کالیبراسیون، جمع احتمال‌ها نمره‌ی غلط می‌دهد (یکی از اشتباه‌های رایج در راهنما).
- mutantهای بدون پوشش قطعاً زنده‌اند و بدون هزینه کنار گذاشته می‌شوند. این مزیت به همه‌ی روش‌ها، از جمله baselineها، داده شده تا مقایسه منصفانه باشد.

کیفیت پیش‌بینی (AUC، mutantهای پوشش‌داده‌شده)

0.656

بین‌پروژه‌ای (LOPO)

0.741

پروژه‌های held-out

گام ۳: نمونه‌گیری فعال و تخمین‌گر تفاضلی

- در هر قدم یک mutant اجرانشده با احتمال q انتخاب و واقعاً اجرا می‌شود.
- mutantهای نامطمئن (p نزدیک ۰٫۵) شانس بیشتری دارند؛ ۳۰٪ یکنواخت تضمین می‌کند هر mutant شانس انتخاب داشته باشد.
- هر انتخاب یک تخمین نااریب از تعداد کشته‌ها می‌دهد: «پیش‌بینی مدل + خطای مدل روی نمونه، وزن‌دار با q/۱».
- اگر مدل خوب باشد، خطا کوچک است و واریانس پایین می‌آید. اگر بد باشد، تخمین باز هم نااریب می‌ماند.

```
q_i ∝ 0.7·sqrt(p_i(1−p_i)) + 0.3·(1/n)

T_j = Σ_executed y + Σ_unexecuted p
        + (y_j − p_j) / q_j

E[T_j] = #killed   (unbiased)
```

گام ۴ و ۵: تطبیق، ترکیب و بازه‌ی اطمینان

- تطبیق: مدل بین‌پروژه‌ای ممکن است برای پروژه‌ی جدید اریب سراسری داشته باشد؛ با برچسب‌های اجراشده دوباره کالیبره می‌شود.
- چون p و q فقط به برچسب‌های قبلی وابسته‌اند، نااریبی حفظ می‌شود.
- ترکیب empirical-Bayes: به پیش‌بینی مدل فقط تا جایی اعتماد می‌کنیم که داده نشان ندهد اریب است. این ترکیب در بودجه‌ی خیلی کم (۵٪) بیشترین سود را دارد.
- بازه‌ی اطمینان از واریانس تخمین‌های T_j به دست می‌آید؛ پوشش ۹۵٪ آن در نتایج بررسی شده است.

```
after each round r = 1..4:
  p ← Platt(logit p_base ; labels so far)

T̂ = mean_j T_j      V = s²(T_j) / n
M  = Σ_executed y + Σ_unexecuted p
w  = V / (V + max(0, (T̂ − M)² − V))
score = (w·M + (1−w)·T̂) / N
CI95  = score ± 1.96·sqrt(V) / N
```

شبه‌کد و پیاده‌سازی

```
def ace(project, budget):
    p = calibrated_cross_project_model(project)
    pool = covered_mutants(project)      # uncovered = survived
    for r in range(4):                   # rounds
        for _ in range(budget // 4):
            q = 0.7*unc(p) + 0.3*uniform  # unexecuted only
            j = sample(pool, q); y[j] = run_pit(j)
            T.append(known_y + sum(p_unknown) + (y[j]-p[j])/q[j])
        p = platt_adapt(p_base, executed, y)
    est, V = mean(T), var(T)/len(T)
    return eb_combine(est, model_pred(p), V), ci(V)
```

- **کد: ace → pmt/estimators.py()**
- همه‌ی پارامترها در configs/default.yaml
- دستور اجرا: python -m pmt all
- تست واحد ناایبی: pytest -q
- عمومی و CSV، Python/mutmut، Java/PIT داده‌ی مصنوعی

نتایج

28%

کاهش خطا نسبت به تصادفی در بودجه‌ی ۵٪

16%

کاهش خطا نسبت به تصادفی در بودجه‌ی ۱۰٪

96%

پوشش بازه‌ی ۹۵٪ ACE-MT (بودجه ۱۰٪)

13%

سهم اجرای تست لازم در بودجه‌ی ۱۰٪ (RQ3)

- RQ1: AUC بین‌پروژه‌ای 0.656 و درون‌پروژه‌ای 0.744. PMT و Seshat-lite بدون اجرا خطای 0.142 و 0.119 دارند.
- RQ2: در مقایسه با تصادفی در ۱۰٪، Wilcoxon p = 0.0032. «فقط مدل» بازه‌ی نامعتبر دارد (پوشش 40%).
- RQ4: τ برای ACE-MT 0.833 و برای تصادفی 0.809؛ PMT در رتبه‌بندی بهتر است (0.872).
- RQ5: مؤثرترین جزء ترکیب empirical-Bayes است (بدون آن MAE = 0.051 در برابر 0.044).



میانگین روی ۱۰ پروژه و ۳۰ تکرار. «فقط مدل» هیچ mutantی اجرا نمی‌کند.